Technical Design: Multi-Source Candidate Data Transformer

1. Pipeline Architecture

The system processes messy, multi-source inputs via a deterministic six-stage data pipeline. Each component handles localized transformations, ensuring that malformed inputs degrade gracefully without cascading failures.

```
[Raw Sources] ──> [Ingest & Extract] ──> [Normalize] ──> [Entity Resolution] ──> [Field Merging & Scoring] ──> [Custom Projection] ──> [JSON Schema Validation]
```

- **Ingest & Extract**: Decoupled parsers read specific source formats (ATS JSON, Recruiter CSV, GitHub API). Instead of mapping directly to the final schema, each parser emits uniform, flat metadata tuples: (field_path, raw_value, source_identity, ingestion_timestamp).
- **Normalize**: Applies specialized cleaning scripts to isolate and standardize data types (phones, dates, locations, and skills).
- **Entity Resolution:** Evaluates identity across incoming records using progressive matching keys to group separate records belonging to the same candidate.
- **Merge & Score:** Collapses grouped records into a single Internal Canonical Record. Clashing fields are resolved using a deterministic source-hierarchy matrix.
- **Custom Projection**: Evaluates the runtime configuration to dynamically mask, slice, and rename fields from the canonical model into the requested shape.
- **Schema Validation:** verifies the structural integrity of the output against the configuration constraints, triggering the configured fallback (omit, null, error) for missing properties.

2. Canonical Schema & Field Normalization Targets

The internal canonical record acts as the definitive source of truth. To track provenance, all primitive values are held inside an internal metadata wrapper.

Internal Meta Container Struct

```
class ProvenanceMetadata(BaseModel):
    source: str
    method: str

class FieldContainer(Generic[T]):
    value: Optional[T] = None
    confidence: float = 1.0
    provenance: List[ProvenanceMetadata] = []
```

Canonical Data Fields

- candidate_id: Standard UUIDv4 string.
- full_name, headline: Wrapped standard string tokens.
- emails, phones, skills: Explicitly typed sets of wrapped objects to enforce unique collections across source extractions.
- location: Nested structural container: { city: FieldContainer[str], region: FieldContainer[str], country: FieldContainer[str] }.
- links: Set of mappings capturing specific keys: { linkedin, github, portfolio, other: List[str] }.
- experience: Array of complex objects: List[{ company, title, start, end, summary }].
- education: Array of complex objects: List[{ institution, degree, field, end_year }].
- overall_confidence: Floating-point scalar score bounded between 0.0 and 1.0.

Field Normalization strategy

- **Phones:** Strips whitespace, hyphens, and alphabetic characters via regular expressions, formatting directly to E.164. If a country code is missing, the system infers it from the candidate's localized country value.
- **Dates:** Converts varying text formats ("Sept 2022", "2022/09", "09-2022") to strict ISO-8601 (YYYY-MM) strings. "Present" or "Current" values map to null while flipping an internal is_current boolean flag to allow precise date arithmetic.
- **Country Codes:** Normalizes location text strings to ISO-3166-1 alpha-2 identifiers via a compiled in-memory lookup dictionary (e.g., "United States", “USA”->US).
- **Skill Tokens:** Cleans engineering terms by applying lowercasing, stripping special characters, and dropping common syntax suffixes (e.g., "React.js", "ReactJS", and "react-js" unify to "react").

3. Entity Resolution & Conflict Merger Logic

Identity Matching Criteria

Candidates extracted from mismatched data sources are grouped using a cascading resolution rule-set:

1. **Primary Key Match:** Exact, case-insensitive string match across extracted emails.
2. **Secondary Key Match:** Identity matching on verified public profiles (e.g., matching a clean github handle string).
3. **Heuristic Validation Fallback:** Intersecting a normalized full_name combined with an overlapping employment duration at the same employer token.

Conflict Resolution Strategy

When conflicting fields emerge across sources, values are selected using a static authority ranking paired with token recency:

$$\textit{Selection Score} = \textit{Source Authority Weight } x \textit{ Recency Factor}$$

Source Classification	Authority Base Weight	Default Extraction Method
ATS JSON Payload	0.95	Direct Mapping
GitHub REST API	0.85	Rest Client Query

Source Classification	Authority Base Weight	Default Extraction Method
Recruiter CSV	0.75	Row parsing
Unstructured Resume (PDF)	0.60	Regex Text Blocks
Recruiter Notes (.txt)	0.45	Heuristic Extraction

- **Scalar Merging:** The single string or numeric value from the source with the highest Selection Score overrides competing fields. If scores tie, the token with the most recent ingestion date is preserved.
- **Array/Complex Object Merging:** Array attributes (Experience, Education) are grouped by unique identifier keys (company + title). If overlapping timelines are found, the inner attributes are merged using the priority table to combine summaries while retaining clear data lineages.
- **Confidence Scoring Model:** The overall_confidence value matches the mathematical average score of all distinct incoming sources contributing to the record, applying a deduction penalty of -0.15 for every missing primary identity descriptor (emails, full_name).

4. Projection Layer & Runtime Config

The pipeline separates internal data storage from dynamic serialization, using the config payload as a transformation map.

- **Path Mapping:** Decodes JSON-path strings and index hooks (emails[0], skills[].name) to fetch nested elements from the internal canonical model.
- **Dynamic Data Masking:** Prunes fields missing from the incoming layout config using deep dictionary comprehensions. Confidence and provenance values are stripped inline unless the config flags include_confidence or explicit logging toggles as True.
- **Missing Field Branching (on_missing):** When data elements map to empty fields, the system executes one of three behaviors:
    - omit: Completely drops the structural dictionary key from the generated output map.
    - null: Emits the dictionary key with an explicit JSON null assignment.
    - error: Interrupts execution by raising an explicit validation error, routing the failed candidate to a data quarantine path.

5. System Edge Cases & Scope Limits

Edge Cases Resolved by Design

- **Conflicting Active Roles:** Source A asserts a candidate left a company in 2025-11, but Source B maps the role as "Present". The system trusts the "Present" timeline if Source B holds an equal or higher authority ranking.
- **Ambiguous Local Numbers:** Parsing a ten-digit string lacking country codes (5105550192). The system scans adjacent location records; if a US city is detected, it automatically appends the +1 prefix.

- **Mismatched Work Titles:** A resume identifies a position as "Software Engineer II" while recruiter notes label it "Fullstack Developer". The system groups the records under the identical company timeline, picks the title from the higher-trust source, and stores the secondary title inside an internal alias list.

Strategic Scope Exclusions Under Time Pressure

- **Semantic Graph Skill Expansion:** Checking whether "Machine Learning" implies "Deep Learning" requires heavy embeddings or graph database evaluations. The pipeline restricts its scope to deterministic string normalization maps.
- **Global Educational Translations:** Mapping international degree classifications or translated university names to domestic patterns is excluded; raw strings are passed along directly if standard dictionary lookups miss.
- **Fuzzy Corporate Parent Matching:** Mapping subsidiaries (e.g., knowing "Google", "DeepMind", and "YouTube" belong to "Alphabet Inc.") is omitted. The normalizer strips standard business suffixes (LLC, Inc., Corp.) and relies strictly on exact string matching.